

Capítulo 1

Caso de estudio: detector de alucinaciones en RAG

El propósito de este capítulo es aplicar las técnicas y modelos descritos previamente con el objetivo de construir un detector de alucinaciones para sistemas RAG usando *machine learning* clásico además de presentar y comparar los resultados con los obtenidos mediante técnicas más sofisticadas y costosas computacionalmente.

Un sistema RAG responde a una consulta en dos fases:

- **Recuperación** (*retrieval*): a partir de la consulta, el sistema busca en una base documental las partes más relevantes y los recupera como contexto.
- **Generación** (*generation*): un modelo de lenguaje redacta la respuesta condicionado al contexto, que llamamos fuente.

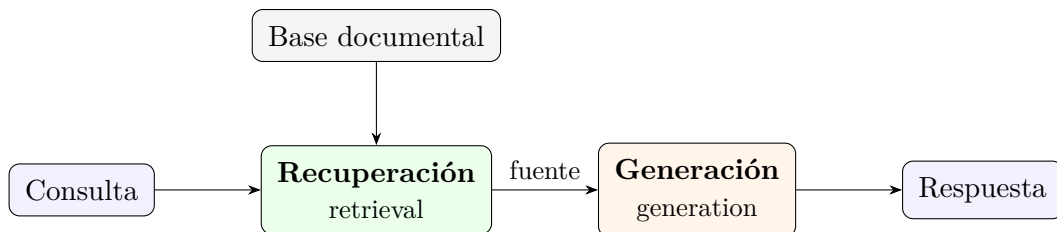


Figura 1.1: Las dos fases de un sistema RAG.

Este trabajo se centra en la segunda fase. Un RAG genera una respuesta en base a una fuente y definiremos una alucinación como una respuesta que no está respaldada por la fuente.

Así, nuestro objetivo es construir un clasificador que, dado el par (respuesta, fuente), decida si la respuesta contiene alguna alucinación, usando exclusivamente aprendizaje automático clásico y características léxicas y estadísticas, sin redes neuronales ni llamadas a LLMs. La motivación es reproducir, a coste mucho menor, lo que hoy hace un evaluador tipo *LLM como juez*, con un modelo barato, interpretable y determinista.

1.1. Presentación de la base de datos

Primero presentamos con detalle la base de datos que utilizaremos en la experimentación. Trabajamos con la versión ya procesada de RAGTruth, una base de datos creada por

Niu et al. en 2024 [4] que consta de 17.790 respuestas de LLMs etiquetadas por humanos a nivel de tramo según contengan o no alucinaciones. Dichas respuestas fueron generadas por seis modelos distintos (GPT-4, GPT-3.5-turbo, Llama-2-7B/13B/70B y Mistral-7B) sobre 2.965 *prompts* y cubren tres tareas de distinta naturaleza:

- **QA:** dada una pregunta el modelo redacta una respuesta. La fuente es texto en prosa.
- **Data2txt:** dado un registro estructurado el modelo redacta una descripción. La fuente son campos estructurados.
- **Summary:** dado un documento el modelo produce un resumen. La fuente es un documento largo.

Se trabajará con la partición oficial de RAGTruth: 15 090 respuestas de entrenamiento y 2 700 de test para poder comparar con el estado del arte.

La base inicialmente está formada por 11 variables entre las que se incluyen la consulta, la calidad de respuesta y la temperatura de muestreo con la que se generó; aunque nosotros solo haremos uso de las cinco siguientes.

- **Context:** Contexto que debe respaldar la respuesta.
- **Output:** Respuesta generada por el modelo de lenguaje
- **Task type:**
 - QA
 - Data2txt
 - Summary
- **Model:** Modelo de lenguaje que genera la respuesta.
 - GPT-4
 - GPT-3.5-turbo
 - Llama-2-7B
 - Llama-2-13B
 - Llama-2-70B
 - Mistral-7B
- **Hallucination labels:** Lista de tramos alucinados con su correspondiente tipo de alucinación. Anotadas por un humano.

Esta última variable marca las alucinaciones. A partir de ella se define la variable objetivo binaria a nivel de respuesta: se etiqueta `label = 1` (alucina) si la respuesta contiene al menos un tramo anotado, y `label = 0` (limpia) en caso contrario. Sobre esta etiqueta se define la *prevalencia*: la fracción de respuestas con alucinación (`label = 1`) en un conjunto dado. Es un término que se usará a lo largo del capítulo, porque varía mucho de una tarea a otra y es lo que da forma al desbalance de clases. El Cuadro 1.1 muestra las primeras filas de la base de datos.

task_type	model	context (fuente)	output (respuesta)	halluc
QA	gpt-4	« <i>Butcher Shop, Hayward, 826 B Street... Phone Number: (510) 889-8690...</i> »	« <i>The phone numbers for several butcher shops are mentioned in the passages...</i> »	[]
Data2txt	gpt-3.5	«{'name': 'Apna Indian Kitchen', 'address': '718 State St', 'city': 'Santa Barbara'}...»	« <i>Apna Indian Kitchen is a local business located at 718 State St in Santa Barbara, CA...</i> »	[]
Summary	gpt-4	« <i>Seventy years ago, Anne Frank died of typhus in a Nazi concentration camp...</i> »	« <i>The Anne Frank House has revealed that Anne Frank and her older sister, Margot...</i> »	[]

Tabla 1.1: Tres primeras filas de RAGTruth

1.1.1. Descriptivo y caracterización de las alucinaciones

Con el objetivo de entender mejor el problema, necesitamos primero entender los datos: qué aspecto tiene una alucinación, con qué frecuencia aparece, de qué tipo suelen ser y si es más propensa a inventar unas cosas que otras. La Figura 1.2 muestra que en general las respuestas tienen en su mayoría alrededor de 100 *tokens* y las tres tareas se reparten de forma equivalente entre entrenamiento y test. Esto es importante porque le da al modelo una aplicación de carácter general, es decir, el modelo no estará entrenado solo para una longitud de respuesta, un tipo de tarea o un modelo concreto.

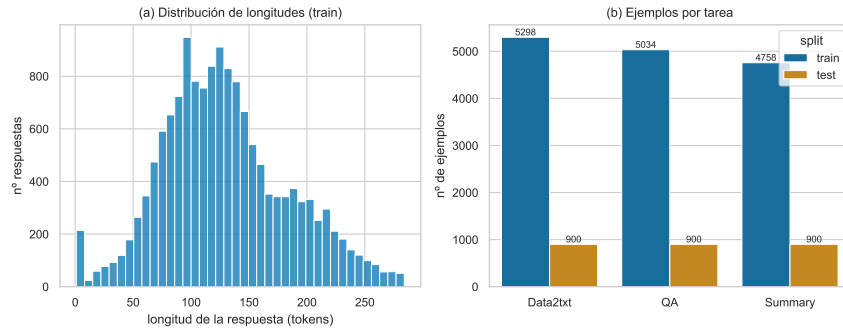


Figura 1.2: Distribución de longitudes de respuesta y número de respuestas por tarea en entrenamiento y test.

La tabla 1.2 recoge un ejemplo de alucinación. Un mismo contexto de Data2txt y dos outputs distintas: una limpia (`label=0`) y otra alucinada (`label=1`). En este caso la respuesta alucinada comete dos tipos de alucinaciones: conflicto directo con la fuente, afirma un dato que la fuente no respalda y además introduce una información sin base, inventa una valoración de 4,5 estrellas que no figura en los datos.

Fuente (datos estructurados)	«... 'categories': 'Gluten-Free, Gastropubs, Vegan, Restaurants, Indian', 'OutdoorSeating': True, ' WiFi ': 'no', 'RestaurantsTakeOut': True ... »
Resp. limpia (label=0)	«Apna Indian Kitchen is a local business located at 718 State St in Santa Barbara, CA. It offers a variety of options for customers, including gluten-free, gastropubs, vegan, and traditional Indian cuisine.»
Resp. alucinada (label=1)	«... The restaurant has a cozy atmosphere with outdoor seating and free WiFi Customers have given the restaurant a 4.5-star rating ... »

Tabla 1.2: Ejemplo de alucinación en RAGTruth.

Frecuencia y desbalance

Casi la mitad de las respuestas de entrenamiento contienen al menos una alucinación, pero el promedio global tapa la estructura real. La Figura 1.3 muestra la proporción por tarea. Data2txt alucina en dos de cada tres respuestas, mientras que QA y Summary se quedan en el 31,1 %. Estamos ante un desbalance heterogéneo por tarea que deberá de ser tratado o bien reequilibrando la base de datos o ajustando el umbral de decisión que separa la respuesta limpia de la alucinada según la tarea.

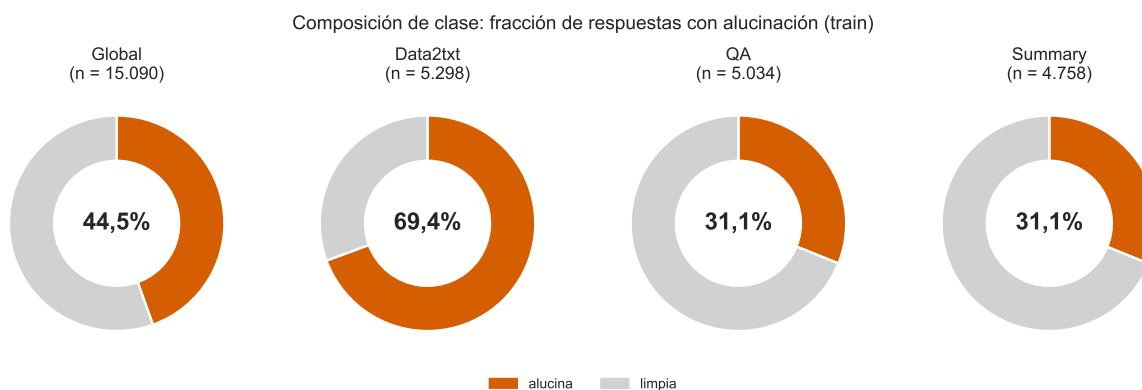


Figura 1.3: Distribución de alucinaciones en entrenamiento.

A la heterogeneidad se suma el desplazamiento entre particiones que vemos en la Figura 1.4.

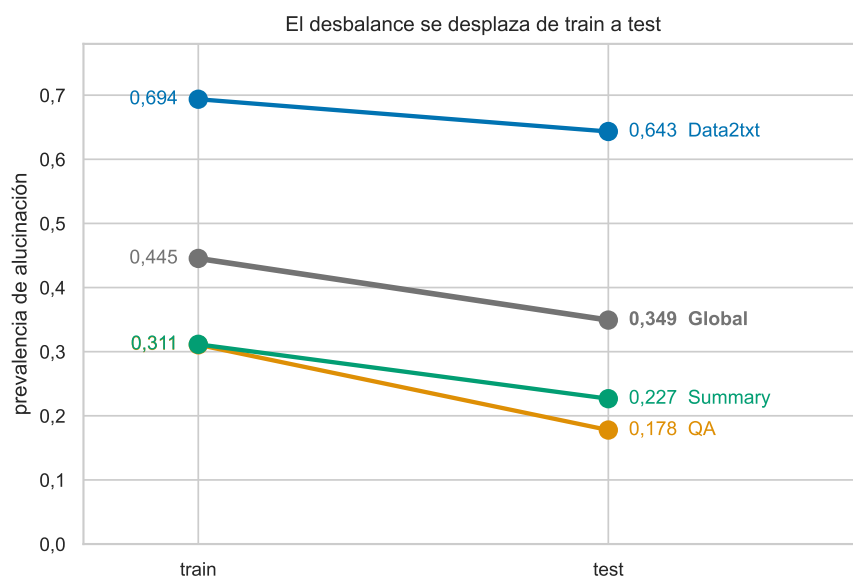


Figura 1.4: Desplazamiento de la prevalencia de alucinación entre entrenamiento y test, por tarea.

Tipos y anatomía del tramo alucinado

RAGTruth etiqueta cada tramo alucinado según dos ejes: si la alucinación es evidente o sutil, y si es información sin base (inventada) o un conflicto directo con la fuente. Ambos tipos ya aparecen en el ejemplo del Cuadro 1.2. La descomposición de cada tarea por tipo de alucinación se recoge en la Figura 1.5. En Summary el 86 % de las alucinaciones son evidentes y, aun así, es la tarea más difícil de detectar. Esto quiere decir que el tipo de alucinación no está relacionado con la dificultad de detección.

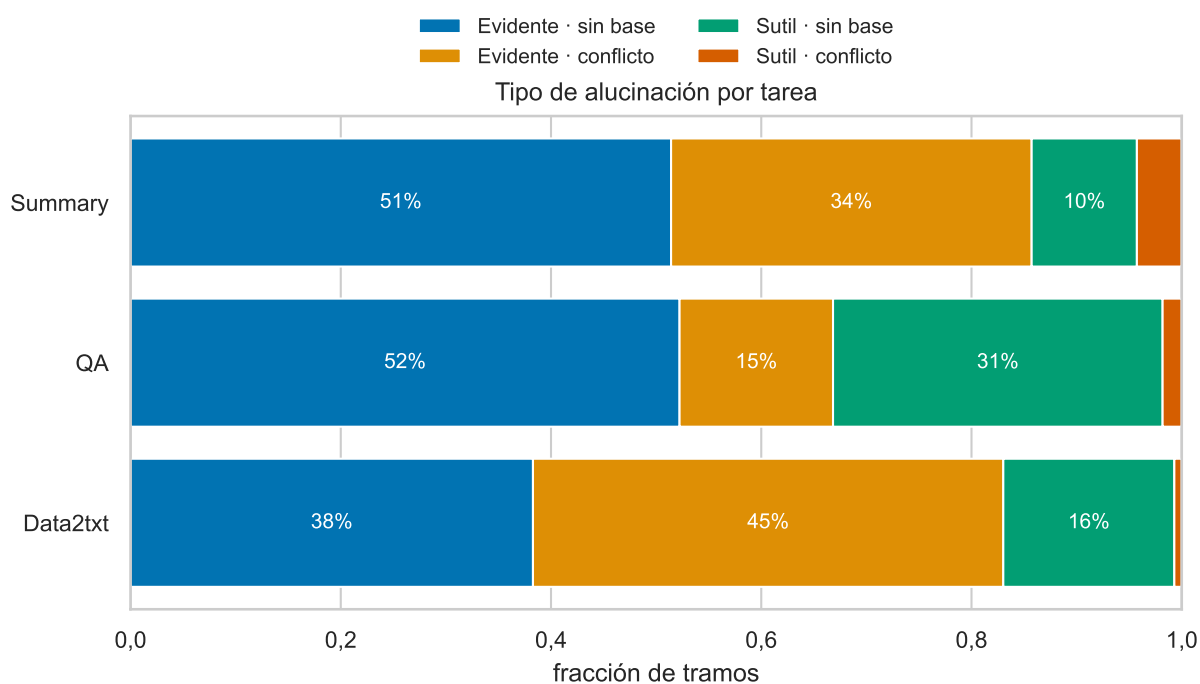


Figura 1.5: Composición de los tipos de alucinación por tarea.

Cuatro de cada diez alucinaciones son una entidad numérica, temporal o nombre propio según la Figura 1.6(a). Los tramos alucinados son cortos, la mayoría no pasa de ocho *tokens* (Figura 1.6b). Esta división es clave para guiar el diseño de características. La parte numérica y temporal tendrá que ser cubierta por variables relacionadas con el solape de números y unidades entre respuesta y fuente, mientras que el texto libre necesitará medidas más orientadas al solape léxico.

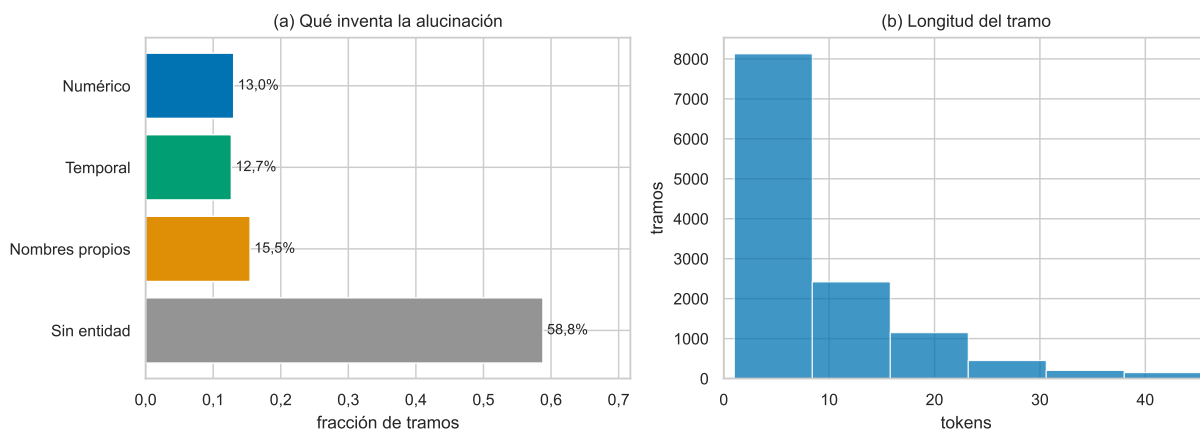


Figura 1.6: Anatomía del tramo alucinado. (a) Categoría de la entidad que introduce la alucinación. (b) Longitud de los tramos alucinados.

Por qué un detector léxico tiene sentido

La anatomía del tramo explica por qué un detector léxico tiene sentido: si casi todas las alucinaciones son números, fechas o nombres propios ausentes en la fuente, medir cuánto del vocabulario de la respuesta aparece en ella basta para delatarlas. El Cuadro 1.2 da el caso canónico: una cifra que nuestro detector marca al instante y que un LLM-juez deja pasar.

Todo este análisis de los datos nos lleva a pensar que un detector léxico tiene sentido. Si casi todas las alucinaciones son números, fechas o nombres propios ausentes en la fuente, medir cuánto del vocabulario del output no aparece en el contexto podría aportar información. El Cuadro 1.2 muestra cómo el ejemplo canónico es capturado por nuestro modelo mientras que un LLM es incapaz de reconocerlo. Esto ocurre por el conocido como sesgo de racionalización propio de modelos de lenguaje. La respuesta inventa la valoración 4,5, que no está en la fuente. El LLM la evalúa por plausibilidad y la racionaliza. El detector clásico la marca por pertenencia a conjuntos

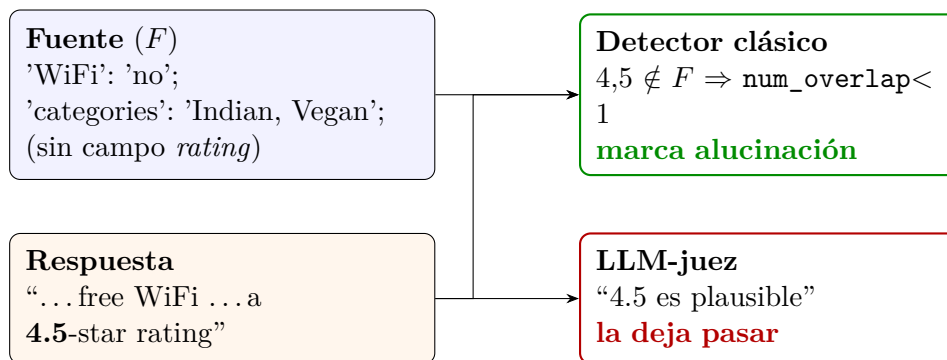


Figura 1.7: Detector basado en *machine learning* clásico frente a LLM aplicado a la misma respuesta. ($4,5 \notin F$).

1.2. Software utilizado

Los datos provienen del corpus RAGTruth ya presentado, descargado de la plataforma *Hugging Face*¹. La experimentación se ha realizado en *Python* 3.12. El código se organiza como una librería instalable de elaboración propia, **ragcheck**, disponible en un repositorio público de GitHub. Todo el flujo es determinista —semilla fija (42) en **numpy**, **scikit-learn** y **xgboost**—, de modo que el mismo código y los mismos datos producen siempre el mismo resultado.

- **Librerías.** El tratamiento de datos y los modelos se apoyan en librerías estándar de *machine learning*: **pandas** y **numpy** para la manipulación de datos, **scikit-learn** y **xgboost** para los clasificadores, **matplotlib** y **seaborn** para las figuras, y **spaCy** para el descriptivo de entidades.
- **Scripts.** Cada tarea tiene su script atómico: **visualization.py** y **spans_analysis.py** para el descriptivo; **log_reg.py**, **knn.py**, **random_forest.py** y **xgb.py** para entrenar y optimizar cada clasificador; y **compare_models.py** para la tabla comparativa final.

1.3. Procesamiento de datos

Los datos de partida son texto, así que el primer paso es transformar cada par (contexto, salida) en un vector numérico de características que capturen indicios de alucinación. Se aplica primero un preprocesado y después se definen las características candidatas; su exploración y selección se abordan en la metodología.

1.3.1. Preprocesado: tokenización

Toda característica se construye sobre una tokenización simple: el texto se pasa a minúsculas y se extraen las secuencias alfanuméricas como *tokens*. Para las características numéricas se usa una variante que conserva los decimales como una sola pieza (“26,2” es un token), necesaria para los *n*-gramas y la consistencia numérica. Sobre esta tokenización se definen los dos conjuntos básicos.

¹<https://huggingface.co/datasets/wandb/RAGTruth-processed>

Definición 1. Dada una respuesta y su fuente, sea R el conjunto de palabras únicas de la respuesta y F el conjunto de palabras únicas de la fuente.

R y F son la base de casi todas las características. La idea es que una respuesta limpia reutiliza el vocabulario de la fuente, mientras que una alucinación introduce términos nuevos o recombina los existentes de forma errónea.

1.3.2. Características candidatas

Se construyen dieciocho características candidatas (Cuadro 1.3), agrupadas en cuatro familias.

- **Solape léxico:** mide cuánto vocabulario comparte la respuesta con la fuente.
- **Numérica y relacional:** busca cifras y unidades inventadas.
- **Nivel frase:** agrega el soporte frase a frase. Una alucinación suele ser una frase sin respaldo que el promedio global tapa, así para delatar la peor frase agregamos con el mínimo.
- **Codificación one-hot de la tarea:** convierte la variable de tarea, que toma tres valores (QA, Data2txt, Summary), en tres columnas binarias, una por tarea, que valen 1 en la tarea correspondiente y 0 en las demás. Así el modelo puede dar un peso distinto a cada tarea y especializar su decisión.

Característica	Definición	Señal que captura
<code>containment</code>	$ R \cap F / R $	precisión léxica (señal dominante)
<code>jaccard</code>	$ R \cap F / R \cup F $	solape simétrico
<code>tfidf_cos</code>	coseno TF-IDF respuesta-fuente	similitud distribucional
<code>num_overlap</code>	fracción de números de R en F	cifras/fechas inventadas
<code>novel_bigram</code>	fracción de bigramas de R no en F	recombinación de pares
<code>novel_trigram</code>	ídem con trigramas	recombinación estricta
<code>num_context</code>	fracción de números con su unidad en F	cambio de unidad/atributo
<code>neg_diff</code>	diferencia de densidad de negación	conflicto de polaridad
<code>answer_len</code>	longitud de la respuesta (tokens)	longitud
<code>sent_cont_*</code>	<code>containment</code> por frase (mín/media/desv/% bajo)	frase peor sostenida
<code>sent_sim_*</code>	similitud por frase (mín/media)	frase menos parecida
<code>task_*</code>	one-hot de la tarea (QA/Summary/Data2txt)	especialización por tarea

Tabla 1.3: Las dieciocho características candidatas. Las filas `sent_cont_*`, `sent_sim_*` y `task_*` agrupan varias columnas (los agregados por frase y las tres indicadoras de tarea), de modo que las doce filas de la tabla suman dieciocho variables.

Se definen en más detalle las cinco básicas de las que se deriva el resto.

Containment (precisión léxica).

$$\text{containment} = \frac{|R \cap F|}{|R|}. \quad (1.1)$$

Fracción de palabras distintas de la respuesta que también están en la fuente. Alto significa que todo lo que afirma la respuesta tiene respaldo léxico; bajo, que introduce palabras nuevas. Mide cuánto de respaldada está la respuesta.

Jaccard.

$$\text{jaccard} = \frac{|R \cap F|}{|R \cup F|}. \quad (1.2)$$

Solape simétrico: mismo numerador que **containment** pero dividido por la unión, así penaliza también el vocabulario de la fuente que la respuesta no usa.

Coseno TF-IDF. El coseno entre los vectores TF-IDF de respuesta y fuente,

$$\cos(u, v) = \frac{\sum_t u_t v_t}{\sqrt{\sum_t u_t^2} \sqrt{\sum_t v_t^2}}, \quad u_t = \text{tf}(t) \cdot \log \frac{N}{\text{df}(t)}, \quad (1.3)$$

que pondera cada término por su rareza: las palabras comunes pesan poco y las específicas, mucho. Mide similitud distribucional, no solo solape de conjuntos.

Solape numérico.

$$\text{num_overlap} = \frac{|N_R \cap N_F|}{|N_R|} \quad (= 1 \text{ si } N_R = \emptyset), \quad (1.4)$$

con N_R, N_F los conjuntos de números de respuesta y fuente. Identifica fechas y cifras inventadas; si la respuesta no tiene números, no hay nada que pueda estar sin respaldo y vale 1.

Longitud de la respuesta. $\text{answer_len} = |\text{tokens}(R)|$, el número de palabras de la respuesta (con repeticiones). Aunque no indica alucinación por sí sola ayuda a contextualizar a las demás, por ejemplo, un containment de 0,8 no significa lo mismo en 3 palabras que en 40.

1.4. Metodología de modelización analítica

La modelización sigue la metodología SEMMA descrita en el marco teórico, aplicada a las variables construidas en el procesamiento de datos. El flujo va del dato preparado a la evaluación en el test (Figura 1.8): se exploran las variables, se fijan las muestras, se depuran y seleccionan, se ajustan los modelos y se valoran sus resultados.

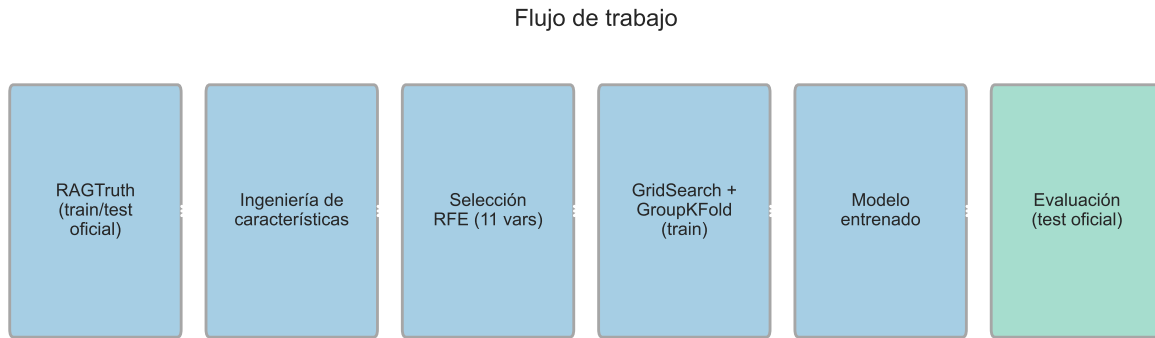


Figura 1.8: Flujo de trabajo: del dato en crudo a la evaluación en el test.

1.4.1. Análisis exploratorio

Los modelos de *machine learning* ponderan más las variables que más información explican y descartan el resto. Un número elevado de variables redundantes solo añade ruido, por eso, antes de seleccionar se comprueban dos cosas: si las características discriminan y cuánta información independiente aportan. Para el primer punto estudiamos la distribución, separada por la variable objetivo, de las quince características continuas; las tres indicadoras de tarea (`task_*`) son binarias y quedan fuera de este análisis. En variables como `containment`, `jaccard` o `num_context` la clase alucinada se desplaza hacia menos solape, como se aprecia en la Figura 1.9.

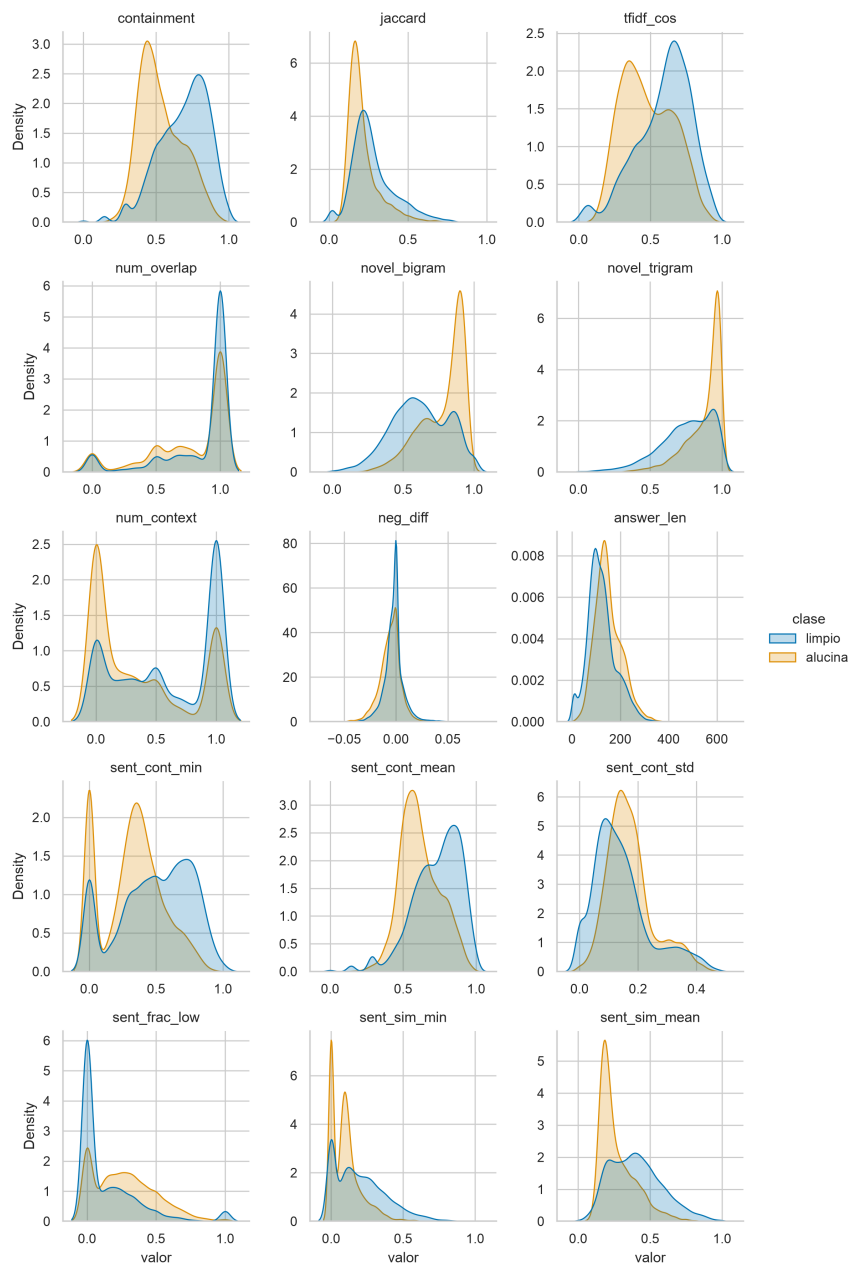


Figura 1.9: Distribución de cada una de las quince características continuas, separada por clase.

Además de discriminar, interesa saber cuánta información independiente aporta cada variable. A continuación se realiza un análisis de correlación entre las variables con el coeficiente de Spearman, elegido por medir asociaciones monótonas —no solo lineales— y por su robustez frente a los valores atípicos, que en nuestras características son frecuentes (respuestas muy largas o solapes extremos). La Figura 1.10 resume las correlaciones entre las dieciocho variables: el bloque de solape léxico está muy correlacionado ($|\rho| > 0,8$); **containment** correlaciona con **tfidf_cos**, **sent_cont_mean**, **sent_sim_mean**, **novel_bigram** y **novel_trigram**, y estas dos últimas son casi su negativo exacto.

Conviene matizar el alcance de esta redundancia. La correlación entre variables predictoras (multicolinealidad) afecta a la interpretación de los coeficientes de la regresión, no a la capacidad predictiva de los modelos; reducir el número de variables, por tanto, no persigue mejores predicciones, sino un modelo más simple e interpretable. Con ese criterio, las

dieciocho variables se agrupan en cuatro ejes de información prácticamente independientes:

1. **Soporte léxico:** `containment`, `jaccard`, `tfidf_cos`, novedad de n -gramas, agregados por frase. Todas miden cuánto se apoya la respuesta en la fuente.
2. **Consistencia numérica:** `num_overlap` y `num_context`, poco correlacionadas con el eje anterior.
3. **Longitud:** `answer_len`, ortogonal al resto.
4. **Polaridad:** `neg_diff`, señal débil pero independiente.

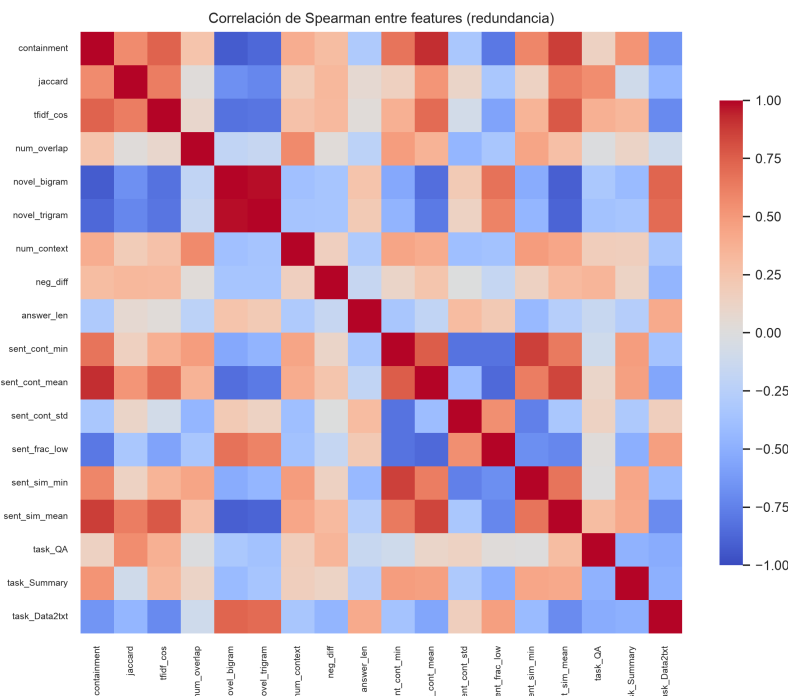


Figura 1.10: Correlación de Spearman entre las dieciocho características. Pocas señales independientes.

1.4.2. Muestreo

Partimos de la división oficial de RAGTruth en dos bloques: **entrenamiento** (15 090 respuestas), sobre el que se ajustan características, hiperparámetros y umbral con validación cruzada interna, y **test** (2 700 respuestas), reservado para la evaluación final. Seguir la partición oficial permite comparar con la literatura sobre el mismo test, y evaluar una sola vez evita el sobreajuste por iteración.

Frente al desbalance de clases no aplicamos remuestreo (sobremuestreo ni submuestreo): en su lugar ponderamos las clases en el entrenamiento (`class_weight`) y ajustamos el umbral de decisión, que la valoración desarrolla. Reservada la partición, queda evitar que los resultados dependan de un corte concreto del entrenamiento.

Para que los resultados no dependan de una única partición usamos validación cruzada de $k = 6$ iteraciones. En nuestra base varias respuestas comparten el mismo documento como fuente, así que aplicamos *GroupKFold* agrupando por `source`: cada documento cae entero en un solo bloque y ninguno aparece a la vez en entrenamiento y validación. Un *KFold* aleatorio filtraría información entre bloques e inflaría el resultado.

1.4.3. Manipulación de datos

La fase de modificación depura las variables y reduce su número. Nuestras características son medidas léxicas y numéricas siempre definidas, así que no hay valores ausentes que imputar; los valores atípicos (respuestas muy largas o solapes extremos) se conservan, pues son informativos y no afectan a los modelos de árbol. El grueso de esta fase es, por tanto, la selección de variables.

Métodos de selección: filtro y envoltante

Los métodos de selección de características se dividen en dos grandes familias [1].

- **Métodos de filtro:** puntúan cada característica por una medida estadística de su relación con la etiqueta (correlación, información mutua, χ^2) y son independientes al modelo utilizado. Son rápidos, pero ignoran las interacciones y la redundancia entre variables.
- **Métodos envoltantes** (*wrapper*): usan el propio modelo como juez. Prueban subconjuntos, los entrenan y se quedan con el que mejor rendimiento obtiene. Son más caros pero más precisos.

Como aquí el problema es justamente la redundancia (sección 1.4.1), usamos un método envoltante.

El método RFE

Recursive Feature Elimination (RFE) [2] es el método envoltante que usaremos. Entrena el modelo con todas las características, se elimina la menos relevante según la importancia interna del modelo, se reentrena y se repite hasta agotar las variables. El orden inverso de eliminación produce un ranking de más a menos importancia.

Trazamos el rendimiento de las métricas (AUC, F1, accuracy, precisión y recall) sobre el entrenamiento, frente al número de variables añadidas en ese orden, y fijamos el tamaño k con el **método del codo** sobre la curva de **F1**: el punto a partir del cual añadir variables deja de aportar o dicho de otra forma, el número de variables óptimas será el k a partir del cual la curva de rendimiento empieza a aplanarse.

Hemos establecido el ranking sobre el XGBoost, la Figura 1.11 muestra cómo el AUC sube de golpe hasta que llegamos a seis variables y luego se aplanan y el F1 alcanza el codo en $k = 11$. Se fija el conjunto en **once variables**. El coseno TF-IDF no entra en la selección, su importancia está por debajo de 0,004, resultado coherente con que la redundancia demostrada con **containment**. En la gráfica precisión y recall se mueven en espejo, cada variable que sube una baja la otra, mientras su media armónica F1 apenas cambia.

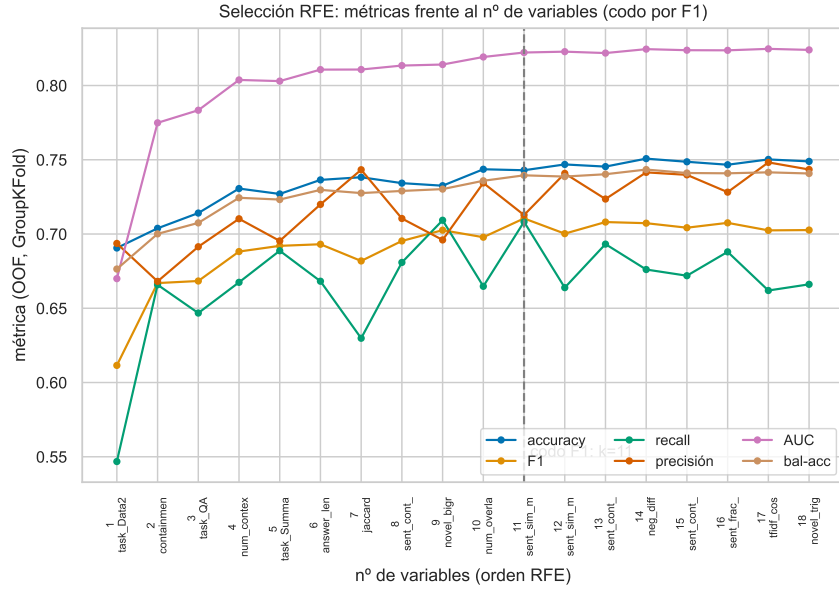


Figura 1.11: Selección mediante RFE (XGBoost): rendimiento frente al número de variables. El codo sobre F1 fija $k = 11$.

Generalizamos el resultado a los demás modelos trazando su AUC, accuracy y F1 al añadir las variables en el mismo orden RFE como se confirma en la Figura 1.12.

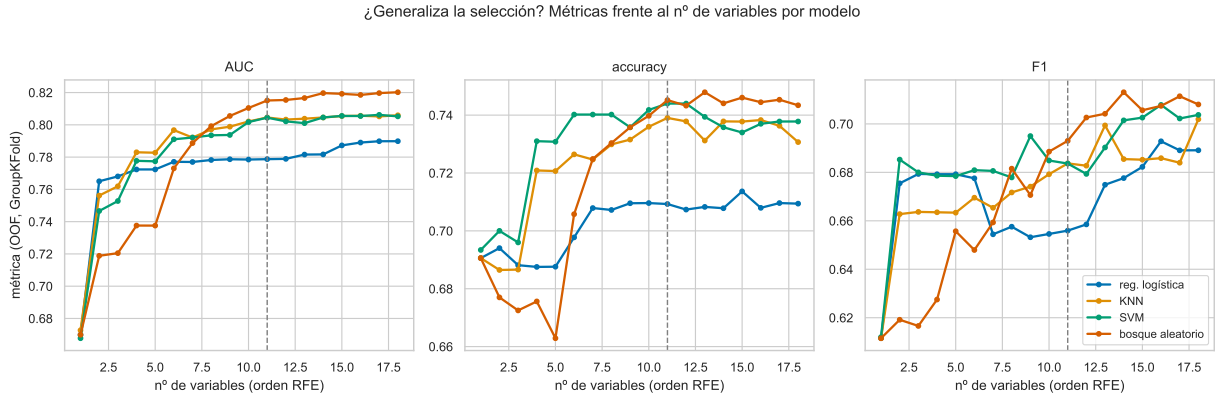


Figura 1.12: AUC, accuracy y F1 del resto de modelos frente al número de variables en orden RFE.

El conjunto de características elegido

Las once variables definitivas, en orden de importancia RFE, son:

`containment`, `task_QA`, `task_Summary`, `task_Data2txt`, `num_context`, `answer_len`, `jaccard`,
`sent_cont_min`, `novel_bigram`, `num_overlap`, `sent_sim_min`.

El conjunto es interpretable y respeta los ejes de la sección 1.4.1: `containment` domina (el eje de soporte léxico); le acompañan la consistencia numérica (`num_context`, `num_overlap`), la recombinación (`novel_bigram`), el soporte por frase (`sent_cont_min`, `sent_sim_min`), la longitud, y la codificación por tarea. Frente a las dieciocho iniciales, el conjunto conserva la señal por clase y reduce la redundancia.

Estas serán las variables sobre las que se basará todo el estudio posterior.

1.4.4. Modelización

Los hiperparámetros de cada modelo se eligen por búsqueda en rejilla: se define una rejilla de combinaciones, se evalúa cada una por validación cruzada *GroupKFold* sobre entrenamiento y validación, y se conserva la de mayor F1. El coste es multiplicativo: optimizar tres hiperparámetros con cuatro valores cada uno y seis *folds* exige $4 \cdot 4 \cdot 4 \cdot 6 = 384$ ajustes del modelo, asumible en CPU para modelos clásicos y una de las ventajas frente a las redes profundas, donde cada ajuste cuesta horas de GPU.

Para cada clasificador se enumeran sus hiperparámetros, con los valores que toma cada uno en la búsqueda, y se optimizan por búsqueda en rejilla con validación cruzada *GroupKFold* de seis iteraciones por **source**: el F1 de cada combinación es la media sobre los seis pliegues, y se conserva la de mayor media. El rendimiento de los modelos ajustados sobre el test se presenta en la valoración.

Regresión logística

La regresión logística, clasificador lineal de referencia, se ajusta con tres hiperparámetros; el solucionador se fija en `liblinear`, que admite las dos penalizaciones:

- **C**: cuanto menor es **C**, más se penalizan los coeficientes grandes y más simple resulta el modelo, lo que reduce el sobreajuste. Se prueban los valores 0,01, 0,1, 1, 10 y 100.
- **penalty**: el tipo de penalización sobre los coeficientes.
 - 11 (Lasso): tiende a anular coeficientes, seleccionando características.
 - 12 (Ridge): encoge todos los coeficientes sin anularlos.
- **class_weight**: determina cómo pesa cada clase en el ajuste.
 - **None**: todas las respuestas pesan igual.
 - **balanced**: cada clase se pondera de forma inversamente proporcional a su frecuencia, de modo que la clase minoritaria (la alucinación) pesa más. Es una manera de compensar el desbalance directamente en el entrenamiento.

En validación cruzada, la mejor combinación es **C** = 0,01,

En validación cruzada, la mejor combinación es **C** = 0,01, **penalty** =12 y **class_weight** =balanced (Cuadro 1.4); lo que marca la diferencia no es el valor de **C** ni la penalización, sino activar el reponderado de clases.

C	penalty	class_weight	F1 (CV)
0,01	l2	balanced	0,675
0,01	l1	balanced	0,674
0,1	l1	balanced	0,674
10	l1	balanced	0,673
100	l1	balanced	0,673
1	l2	balanced	0,673
1	l1	balanced	0,673
10	l2	balanced	0,673
100	l2	balanced	0,672
0,1	l2	balanced	0,672
0,1	l1	None	0,662
0,1	l2	None	0,662
0,01	l1	None	0,660
1	l2	None	0,659
0,01	l2	None	0,658
1	l1	None	0,657
100	l2	None	0,657
100	l1	None	0,657
10	l1	None	0,656
10	l2	None	0,656

Tabla 1.4: Valores F1 (media en CV de 6 iteraciones) para las 20 combinaciones de la regresión logística.

K vecinos más cercanos (KNN)

El clasificador de los k vecinos más cercanos se ajusta con tres hiperparámetros; las características se estandarizan antes, por su sensibilidad a la escala:

- **n_neighbors** (k): número de vecinos que se consultan para decidir la clase. Un k pequeño da fronteras muy locales y ruidosas; uno grande, fronteras más suaves. Se prueban los valores 3, 5, 11, 15 y 25.
- **weights**: cómo se combinan los votos de los k vecinos.
 - **uniform**: todos los vecinos pesan igual.
 - **distance**: cada vecino pesa de forma inversa a su distancia, de modo que los más cercanos influyen más en la predicción.
- **p**: el orden de la distancia de Minkowski con que se miden los vecinos.
 - **p=1**: distancia Manhattan (suma de diferencias absolutas).
 - **p=2**: distancia euclídea.

La rejilla elige $k = 25$, distancia **Manhattan** ($p = 1$) y ponderación por distancia (Cuadro 1.5); la distancia Manhattan supera a la euclídea de forma sistemática, y el rendimiento se satura a partir de $k \approx 15$.

<i>k</i>	<i>p</i>	weights	F1 (CV)
25	1	distance	0,689
15	1	distance	0,689
15	1	uniform	0,688
25	1	uniform	0,686
11	1	uniform	0,685
15	2	distance	0,684
11	1	distance	0,684
25	2	distance	0,684
25	2	uniform	0,684
11	2	distance	0,683
15	2	uniform	0,683
11	2	uniform	0,683
5	1	uniform	0,674
5	1	distance	0,673
5	2	distance	0,672
5	2	uniform	0,671
3	1	uniform	0,663
3	1	distance	0,662
3	2	uniform	0,661
3	2	distance	0,660

Tabla 1.5: Valores F1 (media en CV, 6 iteraciones) para las 20 combinaciones de KNN.

Bosque aleatorio

El bosque aleatorio se entrena con 300 árboles, suficiente para que el promedio se estabilice, y se ajustan tres hiperparámetros:

- **max_depth**: profundidad máxima de cada árbol, es decir, cuántas preguntas encadenadas puede hacer. Árboles poco profundos capturan patrones gruesos; muy profundos, detalles finos a riesgo de sobreajustar. Se prueban 10, 15, 20 y **None** (sin límite: el árbol crece hasta separar el entrenamiento).
- **max_features**: cuántas características se consideran, elegidas al azar, en cada división de un árbol. Reducirlas descorrelaciona los árboles entre sí y suele mejorar el promedio.
 - **sqrt**: la raíz cuadrada del total (aquí ≈ 3 de 11).
 - **log2**: el logaritmo en base dos del total.
- **min_samples_leaf**: número mínimo de muestras que debe quedar en una hoja. Valores mayores producen hojas más pobladas y árboles más suaves, lo que regulariza. Se prueban 1, 2, 4 y 8.

La búsqueda se queda con profundidad máxima 20, **max_features = sqrt** y **min_samples_leaf = 4** (Cuadro 1.6), pero las 32 combinaciones rinden casi igual (entre 0,688 y 0,695): el modelo es muy robusto a sus hiperparámetros.

max_depth	max_features	min_samples_leaf	F1 (CV)
20	sqrt	4	0,695
20	log2	4	0,695
20	sqrt	1	0,695
20	log2	1	0,695
15	sqrt	2	0,694
15	log2	2	0,694
15	sqrt	8	0,694
15	sqrt	4	0,694
sin límite	sqrt	4	0,694
sin límite	log2	4	0,694
20	sqrt	8	0,693
sin límite	sqrt	1	0,693
15	sqrt	1	0,692
10	sqrt	2	0,691
10	sqrt	4	0,690
10	sqrt	1	0,688

Tabla 1.6: Valores F1 (media en CV, 6 iteraciones) del bosque aleatorio: las 16 mejores combinaciones de las 32.

Gradient boosting (XGBoost)

El *gradient boosting* (XGBoost) se entrena con 300 árboles y regularización fijos, y se ajustan los tres hiperparámetros más influyentes:

- **max_depth**: profundidad de cada árbol; controla la complejidad de las interacciones que puede capturar. Se prueban los valores 3, 4, 5 y 6.
- **learning_rate**: tasa de aprendizaje, es decir, cuánto contribuye cada árbol nuevo a la predicción. Una tasa baja aprende más despacio pero generaliza mejor. Se prueban los valores 0,03, 0,05, 0,1 y 0,2.
- **subsample**: fracción de ejemplos, tomada al azar, con la que se entrena cada árbol. Menos del total (0,8) introduce aleatoriedad y regulariza; 1,0 usa todos. Se prueban 0,8 y 1,0.

Gana la combinación profundidad 6, tasa de aprendizaje 0,05 y **subsample** = 0,8 (Cuadro 1.7), pero las 32 combinaciones rinden muy parecido (entre 0,678 y 0,696).

max_depth	learning_rate	subsample	F1 (CV)
6	0,05	0,8	0,696
4	0,1	0,8	0,695
5	0,1	0,8	0,694
4	0,1	1,0	0,694
6	0,03	0,8	0,694
6	0,05	1,0	0,694
3	0,1	0,8	0,694
3	0,2	0,8	0,693
4	0,2	1,0	0,693
6	0,03	1,0	0,692
6	0,1	0,8	0,692
5	0,05	0,8	0,692
3	0,1	1,0	0,691
5	0,05	1,0	0,691
6	0,1	1,0	0,691
4	0,05	0,8	0,691

Tabla 1.7: Valores F1 como media en CV en k=6 iteraciones de XGBoost: las 16 mejores combinaciones de las 32.

1.4.5. Valoración

La valoración presenta el rendimiento de los modelos ajustados sobre el conjunto de test. Cada modelo se evalúa una sola vez, con el umbral de decisión fijado en entrenamiento por el índice de Youden; conviene por tanto explicar antes ese umbral.

El tercer problema es propio del desbalance de nuestra base. El modelo produce un *score* continuo por respuesta, y el **umbral** es el corte que lo convierte en decisión: se considera alucinación si el *score* lo supera. Moverlo desliza el *trade-off* entre precisión y recall, bajarlo captura más alucinaciones a costa de más falsas alarmas (Figura 1.13a). Como la prevalencia de alucinaciones difiere por tarea, el corte óptimo es distinto en cada una: donde la alucinación es rara (QA, Summary) conviene bajar el umbral para no dejarla pasar (Figura 1.13b).

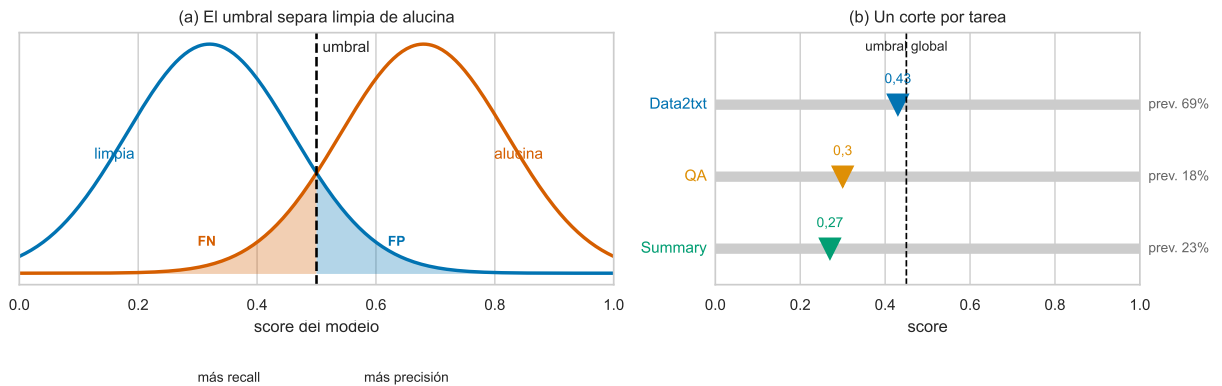


Figura 1.13: (a) El umbral separa las dos clases sobre el score; moverlo intercambia falsos negativos por falsos positivos. (b) Cada tarea opera a una prevalencia distinta, así que su corte óptimo también es distinto.

La Figura 1.14 compara tres criterios para fijar el umbral, todos ajustados en entrenamiento y aplicados al test: el índice de Youden global, el máximo de F1 por tarea y el

máximo de F_2 por tarea. Si fijamos el umbral por tarea recuperamos falsos negativos que el umbral global deja pasar, a costa de perder precisión. Pasamos de 285 falsos negativos a 125.

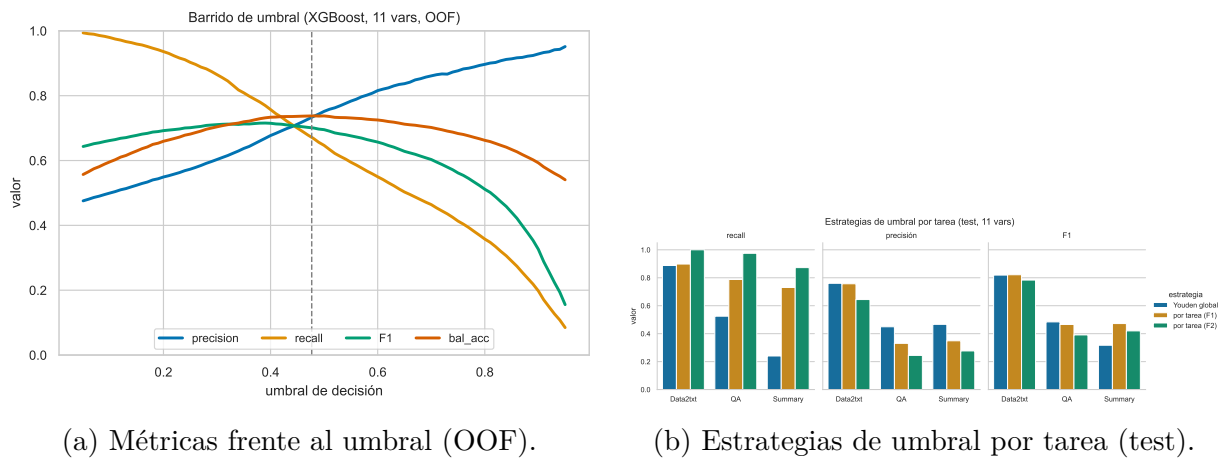


Figura 1.14: Efecto del umbral de decisión. Precisión, recall y F1 al mover el umbral (a). Las tres estrategias de umbral por tarea sobre el test (b).

Con el umbral de Youden fijado en entrenamiento, el Cuadro correspondiente a cada modelo recoge sus métricas sobre el test.

Regresión logística. Es el modelo más débil: su AUC (0,798) es el único que no llega a 0,80, coherente con que solo puede trazar fronteras lineales.

Regresión logística	
Precisión	0,594
<i>Recall</i>	0,726
Valor F1	0,654
Accuracy	0,731
AUC	0,798

Tabla 1.8: Precisión, *recall*, F1 y AUC de la regresión logística en el test.

KNN. Separa razonablemente bien (AUC 0,826), aunque no destaca en precisión ni en recall.

KNN	
Precisión	0,646
<i>Recall</i>	0,695
Valor F1	0,669
Accuracy	0,760
AUC	0,826

Tabla 1.9: Precisión, *recall*, F1 y AUC de KNN en el test.

Bosque aleatorio. Alcanza un AUC de 0,833, de los más altos.

Bosque aleatorio	
Precisión	0,656
<i>Recall</i>	0,699
Valor F1	0,677
Accuracy	0,767
AUC	0,833

Tabla 1.10: Precisión, *recall*, F1 y AUC del bosque aleatorio en el test.

Gradient boosting (XGBoost). Obtiene el mejor AUC de los cuatro modelos, 0,837.

XGBoost	
Precisión	0,673
<i>Recall</i>	0,698
Valor F1	0,685
Accuracy	0,776
AUC	0,837

Tabla 1.11: Precisión, *recall*, F1 y AUC de XGBoost en el test.

1.5. Discusión

1.5.1. Conclusiones de la evaluación. Modelo ganador

Para elegir el mejor modelo usamos como criterio el promedio entre F1 y AUC, que equilibra la separación pura (AUC) con el equilibrio entre precisión y *recall* y por tanto el rendimiento al umbral (F1). El Cuadro 1.12 recoge las métricas de los cuatro modelos. Las diferencias son pequeñas y todos poseen un AUC en torno a 0,80.

Modelo	AUC	F1	precisión	recall	accuracy	bal-acc	prom. F1–AUC
XGBoost	0,837	0,685	0,673	0,698	0,776	0,758	0,761
Bosque aleatorio	0,833	0,677	0,656	0,699	0,767	0,751	0,755
KNN	0,826	0,669	0,646	0,695	0,760	0,745	0,748
Reg. logística	0,798	0,654	0,594	0,726	0,731	0,730	0,726

Tabla 1.12: Comparación de los cuatro modelos en el test (once variables, umbral de Youden fijado en entrenamiento). Ganador por promedio F1–AUC: XGBoost.

XGBoost tiene el mejor AUC y un F1 competitivo, de modo que el promedio de AUC y F1 lo sitúa primero. Los cuatro modelos, en todo caso, quedan muy cerca.

La diferencia está en cómo reparte cada uno la precisión y el recall. El bosque aleatorio y la regresión logística detectan más alucinaciones a cambio de más falsas alarmas: mayor recall, menor precisión. XGBoost es el más equilibrado de los cuatro. En detección de alucinaciones, el falso negativo —una invención que pasa como fiable y llega al usuario sin control— cuesta más que la falsa alarma, que solo obliga a una revisión; por eso el recall es la métrica que prima, y ese mayor recall inclina la balanza hacia los modelos de árbol. La regresión logística, además, es la única que no llega a un AUC de 0,80: su frontera lineal se queda corta donde el resto, no lineales, sí llegan.

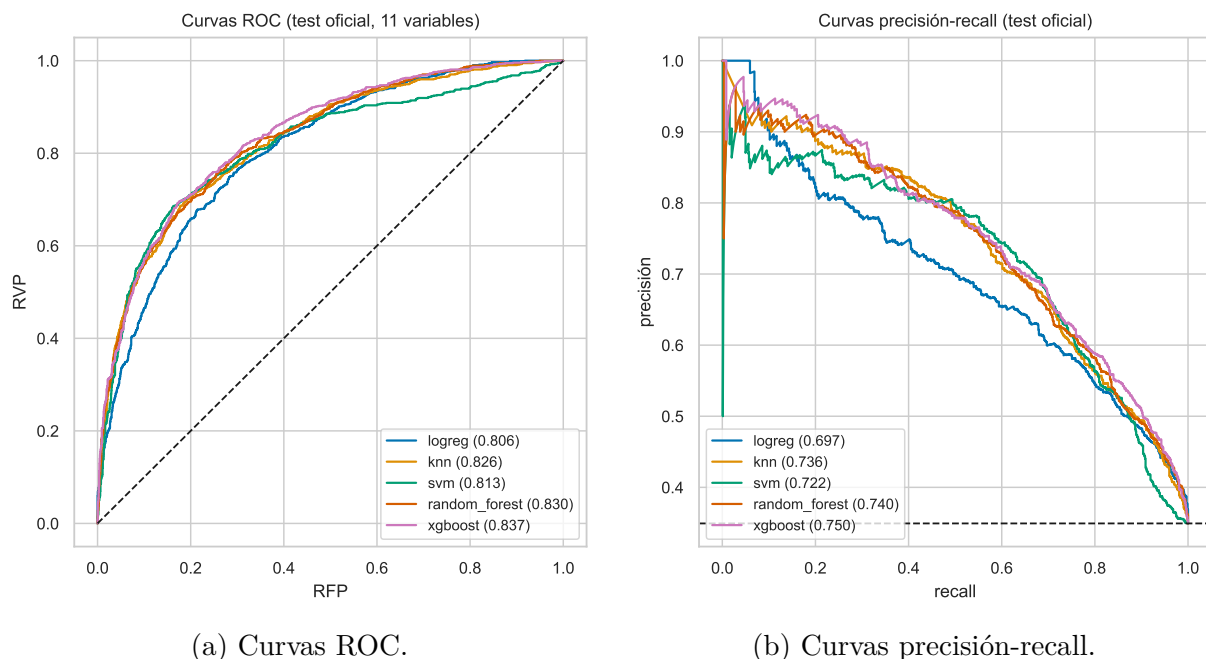


Figura 1.15: Métricas de evaluación de los cuatro modelos sobre el test.

El AUC global de XGBoost (0,837) promedia tres tareas con una distribución de la variable respuesta muy dispar. El desglose por tarea de la siguiente tabla separa dónde el detector rinde bien y dónde falla sistemáticamente. El umbral por tarea recorta los falsos negativos de 285 a 125, sobre todo en Summary (146 $\rightarrow$ 46) y QA (76 $\rightarrow$ 27); a costa de ello, la precisión, la *accuracy* y el F1 agregados bajan.

Tarea	prevalencia	AUC	F1	F1*	recall	recall*	accuracy	accuracy*	FN	FN*
Data2txt	0,643	0,786	0,824	0,820	0,891	0,910	0,754	0,742	63	52
QA	0,178	0,817	0,490	0,463	0,525	0,831	0,806	0,657	76	27
Summary	0,227	0,702	0,358	0,454	0,284	0,775	0,769	0,578	146	46
Global	0,349	0,837	0,685	0,640	0,698	0,867	0,776	0,659	285	125

Tabla 1.13: XGBoost por tarea sobre el test. Las columnas con * usan el umbral optimizado por tarea. En la fila *Global*, las métricas con * se agregan aplicando a cada respuesta el umbral de su tarea y midiendo sobre el conjunto completo (*pooled*); FN* es el total.

Summary es la principal limitación, y no por el umbral sino por la señal: su AUC de 0,70 es el más bajo porque un resumen reutiliza el vocabulario del documento, y el tramo alucinado comparte palabras con la fuente aunque la alucinación sea evidente. Dentro de Summary, la Figura 1.16 muestra que los falsos negativos son justo los tramos de mayor solape con la fuente: ninguna característica léxica es capaz de separar el resumen limpio del alucinado cuando ambos recombinan el mismo vocabulario. Es la limitación del enfoque léxico, no un fallo del modelo.

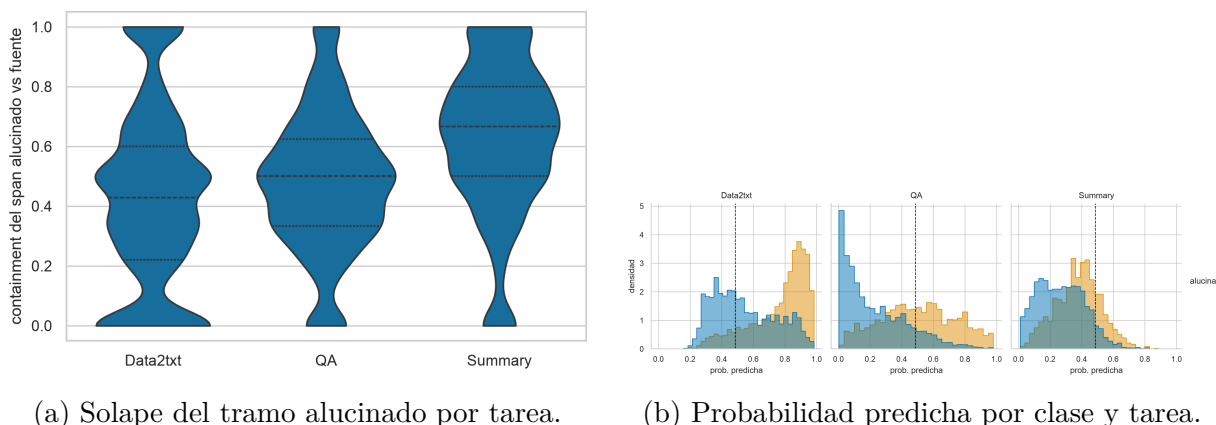


Figura 1.16: La limitación de Summary es el camuflaje léxico.

QA plantea el problema contrario: discrimina bien (AUC 0,82), pero su baja prevalencia hace que un umbral global resulte demasiado exigente y reduzca el recall, dejando sin detectar buena parte de las alucinaciones pese a ordenarlas correctamente. La Figura 1.17 cuantifica ese compromiso: exigir un recall del 80 % apenas cuesta precisión en Data2txt, pero la reduce drásticamente en QA y Summary.

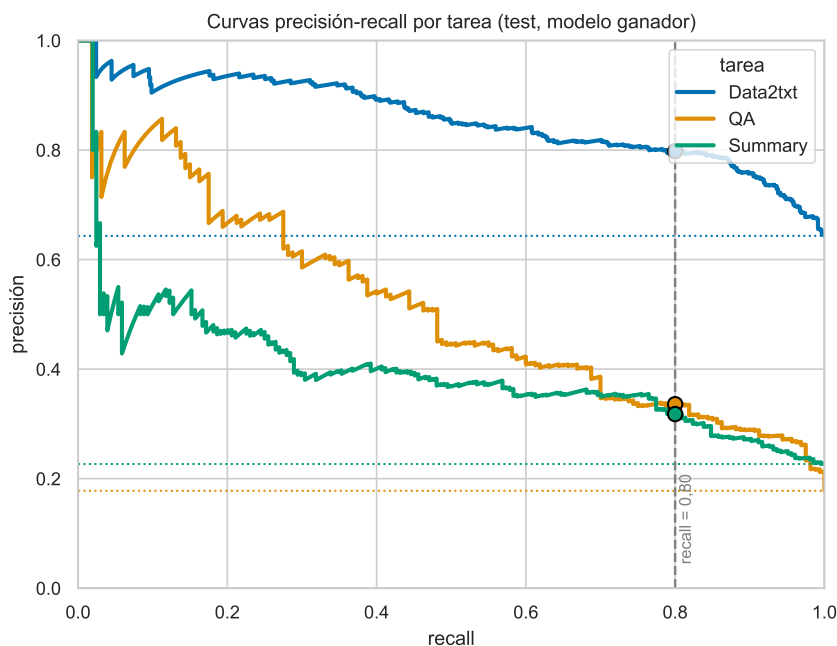


Figura 1.17: Curvas precisión-recall por tarea con la línea base en y =prevalencia de alucinaciones en dicha tarea.

La solución está en fijar un umbral por tarea —el que maximiza el F1 de cada tarea sobre el entrenamiento, no el de Youden global—, reconociendo así su distinta prevalencia: con ello los falsos negativos totales bajan de 285 a 125, sube el recall de QA y mejora el F1 de Summary. Las matrices de confusión por tarea de la Figura 1.18 muestran cómo bajo el umbral por tarea, la celda de falsos negativos (las alucinaciones que el modelo no detecta) encoge en QA y Summary frente al umbral global.

Matrices de confusión por tarea (XGBoost, 11 vars, test)

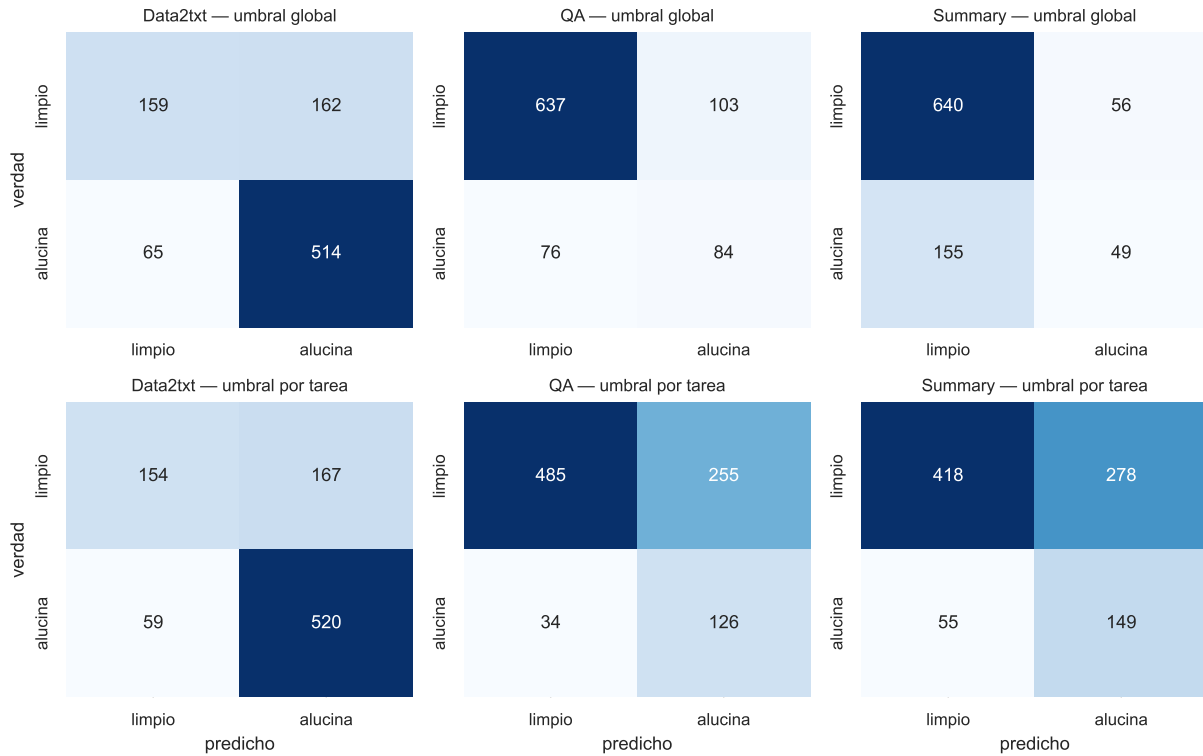


Figura 1.18: Matrices de confusión por tarea (XGBoost, test). Fila superior: umbral global; fila inferior: umbral por tarea. La detección de alucinaciones (VP) crece en QA y Summary.

1.5.2. Comparación con el estado del arte

Los métodos de referencia

El estado del arte sobre RAGTruth es enteramente neuronal y se agrupa en tres familias:

- **LLM-juez** (GPT-4-turbo, GPT-4o): un modelo de lenguaje grande emite el veredicto a partir de un *prompt*, sin entrenarse. No es determinista y cuesta del orden de un céntimo y varios segundos por evaluación.
- **Detectores *fine-tuned*** (Llama-2-13B, Lynx-8B/70B, RAG-HAT): ajustan un LLM completo sobre RAGTruth. Máximo rendimiento, a cambio de miles de millones de parámetros y GPU.
- **Encoders ligeros** (Luna, LettuceDetect, 150–396 M sobre ModernBERT): un punto intermedio, también neuronal y sobre GPU.

El siguiente Cuadro los ordena por tamaño y coste. La mayor diferencia entre estos métodos y nuestro detector es el coste computacional. Frente a miles de millones de parámetros y GPU, nuestro modelo clásico usa del orden de 10^4 parámetros (unos 300 árboles someros sobre 11 características), corre en CPU y es determinista.

Método	Tipo	Parámetros	Hardware	Coste/eval
GPT-4-turbo (juez)	LLM-juez	$\sim 10^{12}$ (est.)	API	$\sim 10^{-2} \text{€}$
Luna	encoder ligero	$\sim 0,4 \text{ B}$	GPU	$\sim 10^{-4} \text{€}$
LettuceDetect-large	encoder (ModernBERT)	0,396 B	GPU	$\sim 10^{-4} \text{€}$
Fine-tuned Llama-2-13B	LLM fine-tuned	13 B	GPU	$\sim 10^{-3} \text{€}$
Lynx-8B / 70B	LLM fine-tuned	8–70 B	GPU(s)	$10^{-3} \text{--} 10^{-2} \text{€}$
RAG-HAT	LLM fine-tuned	(grande)	GPU	$\sim 10^{-3} \text{€}$
Este trabajo	ML clásico	$\sim 10^4$	CPU	≈ 0

Tabla 1.14: Los métodos de referencia sobre RAGTruth, por tipo, tamaño, hardware y coste por evaluación.

Todos, con excepción de Lynx, evalúan sobre el split train/test definido por RAGTruth y a nivel de respuesta, de modo que las cifras son comparables. Como la literatura reporta mayoritariamente F1 y accuracy haremos la comparación sobre esas métricas. La siguiente tabla muestra una comparación de los métodos del estado del arte en base a F1.

Método (F1 nivel respuesta, %)	QA	Data2txt	Summary	Global
GPT-4-turbo (juez)	45,6	78,3	47,6	63,4
Luna (encoder ligero)	51,3	75,9	52,5	65,4
LettuceDetect-base (150M)	65,5	87,9	50,5	76,1
Fine-tuned Llama-2-13B	68,2	88,1	59,1	78,7
LettuceDetect-large (396M)	70,2	88,5	59,7	79,2
RAG-HAT (mejor)	74,8	91,6	67,6	83,9
Este trabajo (clásico)	46,3	82,0	45,4	68,5

Tabla 1.15: F1 por tarea sobre RAGTruth, recopilado por Kovács et al. [3] con RAG-HAT [6] como cota alta. Nuestro F1 por tarea usa el umbral por tarea; el global (68,5) es el F1 pooled con el umbral global (con umbral por tarea el global baja a 64,0). Son dos puntos de operación del mismo modelo.

La otra métrica que reporta la literatura es la accuracy *global* (no por tarea). El Cuadro 1.16 la recoge para la familia Lynx:

Método	Accuracy global (%)
GPT-3.5-turbo	50,7
Este trabajo (clásico)	77,6
Claude-3-Sonnet	79,1
Lynx-8B	80,0
Lynx-70B	80,2
GPT-4o	84,3

Tabla 1.16: Accuracy *global* sobre el subconjunto de RAGTruth incluido en HaluBench, recopilada por Lynx [5].

Nuestro 77,6 % queda a la altura de modelos mucho mayores, entre GPT-3.5-turbo y Claude-3-Sonnet. Cabe mencionar que ese número es el obtenido fijando el **umbral a**

nivel global. Aquí reaparece el compromiso del umbral: fijar un umbral por tarea recupera falsos negativos y mejora el F1 por tarea de Summary, pero baja la accuracy global a 65,9% porque acepta más falsos positivos. O priorizamos la accuracy global o priorizamos la detección por tarea.

Para cerrar el capítulo situamos el detector clásico haciendo un balance entre calidad, precio y complejidad. En **calidad**, superamos en F1 global al juez GPT-4-turbo (63,4) y al encoder Luna (65,4), y quedamos entre diez y quince puntos por debajo de los detectores neuronales (79–84). En **precio y complejidad** nuestro modelo es el ganador. El patrón por tarea es común a todos: Data2txt fácil, QA y Summary mucho más difíciles. Al final nos encontramos con el mismo problema que modelos mucho mayores.

Bibliografía

- [1] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 2003.
- [2] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46(1–3), 2002.
- [3] Ádám Kovács and Gábor Recski. LettuceDetect: A hallucination detection framework for RAG applications. *arXiv preprint arXiv:2502.17125*, 2025.
- [4] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, KaShun Shum, Randy Zhong, Juntong Song, and Tong Zhang. RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [5] Selvan Sunitha Ravi, Bartosz Mielczarek, Anand Kannappan, et al. Lynx: An open source hallucination evaluation model. *arXiv preprint arXiv:2407.08488*, 2024.
- [6] Juntong Song, Xingguang Wang, Juno Zhu, et al. RAG-HAT: A hallucination-aware tuning pipeline for LLM in retrieval-augmented generation. In *Proceedings of EMNLP 2024 (Industry Track)*, 2024.